

Day 3: RDBMS DESIGN ASSIGNMENT 1

Entities:

Master tables:

1. Member:

- Member_id - serial primary key
- Name - varchar(100) not null , check trim(name)<>'')
- Phone -char(10) not null , unique , check('^[0-9]{10}\$')
- Is_golden - boolean not null default false (ie he is a bronze)
- Active_rental_count int not null default 0
- Favourite_category_id - smallint can be null
- Foreign key is category id form category table

2. Category:

- Cat_id primary key constraint smallserial
- Name not null unique check it trim is not empty.

3. Movie :

- Movie_id - serial primary key
- Title varchar 100 not null , check title not empty
- Categord_id -> smallint not null , foreign key on category table, on update cascade , on delete restrict
- Format - enum in vhs,vcd,dvd and not null

4.Dependent :

- dependent_id - serial primary key,
- Memeber_id int not null , foreign key in member table , on delete cascade
- Active rental cnt - int not null default 0
- Check (active rental cnt >=0 and <=1)
- dependent name not empty by trim

Transactional table :

5. Rental

- rental id - serial primary key,
- Member id - int foreign key from member, nullable(bcos any one only can rent) , check only one of them is not null

- Dependent id - int foreign key from dependent nullable
- Movie_id - int not null ,foreign key from movie
- Rented at - timestamp default current_timestamp
- Returned at - timestamp null , check if returned > rented ie valid.

Relationships :

- One category can have many movies
- One movie can only have one category
- One member can have many dependents
- One dependent belongs to only one member.
- One member can have many rentals
- One rental is linked to one member
- One movie can have many rentals

DDL Queries (CREATE) :

Category table:

```
create table category (
  category_id smallserial,

  name varchar(50) not null,

  constraint pk_category primary key (category_id),
  constraint uq_category_name unique (name),
  constraint chk_category_name_not_empty check (trim(name) <> '')
);
```

Movie table:

```
create type movie_format_enum as enum ('VHS','VCD','DVD');

create table movie (
  movie_id serial,

  title varchar(100) not null,
  category_id smallint not null,
  format movie_format_enum not null,

  constraint pk_movie primary key (movie_id),
```

```
constraint fk_movie_category
foreign key (category_id)
references category(category_id)
on update cascade
on delete restrict,

constraint chk_movie_title_not_empty
check (trim(title) <> '')
);
```

Member table:

```
create table member (
member_id serial,

name varchar(100) not null,
phone char(10) not null,
is_premium boolean not null default false,
favorite_category_id smallint,

constraint pk_member primary key (member_id),

constraint uq_member_phone unique (phone),

constraint chk_member_phone
check (phone ~ '^[0-9]{10}$'),

constraint fk_member_fav_category
foreign key (favorite_category_id)
references category(category_id)
on delete set null,

constraint chk_member_name_not_empty
check (trim(name) <> '')
);
```

Dependent table :

```
create table dependent (
```

```
dependent_id serial,  
  
member_id int not null,  
name varchar(100) not null,  
  
constraint pk_dependent primary key (dependent_id),  
  
constraint fk_dependent_member  
foreign key (member_id)  
references member(member_id)  
on delete cascade,  
  
constraint chk_dependent_name_not_empty  
check (trim(name) <> "")  
);
```

Rental Table :

```
create table rental (  
    rental_id serial,  
  
    member_id int,  
    dependent_id int,  
    movie_id int not null,  
  
    rented_at timestamp not null default current_timestamp,  
    returned_at timestamp default null,  
  
    constraint pk_rental primary key (rental_id),  
  
    constraint fk_rental_member  
    foreign key (member_id)  
    references member(member_id)  
    on delete cascade,  
  
    constraint fk_rental_dependent  
    foreign key (dependent_id)  
    references dependent(dependent_id)  
    on delete cascade,
```

```
constraint fk_rental_movie  
foreign key (movie_id)  
references movie(movie_id)  
on delete restrict,
```

```
constraint chk_rental_actor  
check (  
    (member_id is not null and dependent_id is null)  
    or  
    (member_id is null and dependent_id is not null)  
),
```

```
constraint chk_rental_dates  
check (returned_at is null or returned_at >= rented_at)  
);
```